- Feature Engineering
- Using String Indexing

On this page

Was this page helpful?Â

Using String Indexing

When scoring an event, models can take into consideration information stored in strings. For that, first you must generate a new numerical field (an index) from the string field, so that you can use the index as a feature in the model.

To index a string field, use the Data Science API (DS-API) of Pulse to create an indexer. An indexer allows to obtain the index value of the string field.

This page describes how to create new features based on a string field for which an indexer was already created.

Prerequisiteâ

The steps described in this topic presume that a data scientist created an indexer for the string field that you want to use. The indexer is a list in Pulse, created with the frequency indexing algorithm. You can find the indexers in Pulse at the bottom of the list of lists, prefixed with `{StringIndexer}` .

Learn how to use the DS-API to create an indexer in the JupyterLab notebook provided with the Feedzai package.

Using an indexed string field in a Pulse planâ

To use the indexed value of a string field, you must add it to the schema of the data source. For that, use a map operator that retrieves the index value from the indexer and adds it to the event. You can access the index in the map operator by using the user-defined function `FrequencyIndexerUDF(indexer_name, input_field_name)` . The following table shows the obligatory parameters you must provide when calling this UDF.

Parameter	Type	Description
indexer_name	String	The indexer_name to be used here is the one defined when the indexer is created. Request this information from the data scientist that created the indexer.
input_field_name	Categorical	The data source field that you want to use.

Exampleâ

The following snippet returns the index value for the merchant_id field of the event:

```
FrequencyIndexerUDF.transform("v1_merchant_indexer", merchant_id)
```

Advanced usage: clippingâ

Pulse's string indexer uses frequency indexing. The frequency index assigns a ranking to each value of the indexed field. The different values are called categories. The more times a string or category shows up in the training data set, the higher its ranking. Example:

If there is too much information stored in the indexer (i.e. too many categories), the new feature might cause overfit in the machine learning model. To limit that, you can use fewer categories through clipping.

You can use clipping in two different stages:

- **When you create the indexer**, i.e. you don't include less frequent categories in the indexer at all.

  The default clipping of DS-API uses 10000 categories as default for the maximum of categories stored in the indexer, and minimum of 100 samples per category. If there are more than 10000 categories, the less frequent ones are dropped. If any category has less than 100 observations, it is also dropped.

  In the documentation of the DS-API included in the JupyterLab notebook, you can read how to clip the indexer when you create it.

- **When you use the indexer in Pulse to retrieve the ranking for a string**, i.e. you ignore less frequent categories at lookup.

See the next section of this topic to learn how to influence the index clipping when you are retrieving the index values.

Clipping indexers in Pulseâ

When you use the indexer to retrieve the ranking for a string, Pulse's default clipping ignores the less frequent categories of the indexer so that maximum 90% of the indexer's categories are taken into consideration, but minimum 10 categories.

The following table shows the two optional parameters that you can use to control index clipping for string fields in Pulse:

Parameter	Type	Description	Default value	Value range
clipping_max_categories_percentage	Float	Uses only the given percentage of categories from the indexer. For ignored categories and for other unknown input values, the indexer returns a default value, which is the number of categories in the clipped indexer.	90.0	[0.0, 100.0]
clipping_categories_limit	Int	Clips the least frequent categories until leaving only the defined number of categories, or until the clipping_max_categories_percentage criterion is met.	10	>= 1

With the clipping parameters, the full signature of the UDF is:

```
FrequencyIndexerUDF.transform(indexer_name, input_field_name,
clipping_max_categories_percentage, clipping_categories_limit)
```

Examples for clippingâ

The following examples show how to use the parameters of the FrequencyIndexerUDF:

1. To use all the indexer categories, use the following expression in the map operator:

```
FrequencyIndexerUDF.transform("v1_merchant_indexer", merchant_id, 100)
```

1. To eliminate the smaller categories in the indexer until only 16.5% of the indexer's categories remain, or only 20 categories remain:

```
FrequencyIndexerUDF.transform("v1_merchant_indexer", merchant_id, 16.5, 20)
```